

1. RA1. ANÁLISIS DEL SECTOR Y NECESIDADES

1.1 CONTEXTO EMPRESARIAL O SOCIAL

El presente proyecto se contextualiza en el sector de las instituciones educativas, concretamente en centros de Educación Secundaria Obligatoria (ESO), Bachillerato y Formación Profesional (FP) que dependen de infraestructuras digitales para su gestión diaria y comunicación. En la actualidad, el ecosistema tecnológico de estos centros está compuesto por páginas web institucionales estáticas y Entornos Virtuales de Aprendizaje (LMS) consolidados en el mercado como Moodle (utilizado ampliamente en plataformas autonómicas como las Aulas Virtuales), Google Classroom o Microsoft Teams.

A nivel organizativo, estas herramientas cumplen con los estándares de distribución de tareas; sin embargo, adolecen de un enfoque centrado en la experiencia del usuario final: el estudiante. Interfaces rígidas como las de Moodle presentan arquitecturas densas, estructuras jerárquicas complejas y un diseño netamente corporativo o administrativo que genera una barrera invisible de desinterés en el alumnado más joven. El contexto socioeducativo actual demanda herramientas que asimilen las dinámicas de consumo digital de las nuevas generaciones, caracterizadas por la interactividad y la gratificación visual inmediata. Por lo tanto, la pertinencia de esta propuesta radica en modernizar radicalmente la experiencia de usuario dentro de la institución escolar, transformando la interacción con la infraestructura digital del centro en un entorno unificado y gamificado que estimule el seguimiento académico activo.

1.2 NECESIDAD DETECTADA

La necesidad identificada en este proyecto no responde a una apreciación subjetiva o aislada, sino a una deficiencia estructural en la usabilidad y la arquitectura de la información de las plataformas académicas vigentes. Herramientas como las aulas virtuales tradicionales sufren de una notable dispersión de contenidos: las fechas de exámenes se ubican en calendarios independientes, las tareas académicas se aglutinan en listados de texto monótonos y los avisos institucionales se aíslan en foros de difícil acceso. Esta falta de fluidez visual provoca un fenómeno de rechazo y agobio en el estudiante, quien percibe el entorno digital como una carga puramente burocrática en lugar de un soporte de aprendizaje.

El impacto de esta problemática en el entorno descrito es directo: pérdida de entregas por confusión en las fechas, desvinculación de los canales oficiales de comunicación del centro y una total dependencia de los tutores o profesores para recordar tareas básicas. Abordar esta carencia mediante una aplicación interactiva resulta crítico. Al unificar los datos dispersos de la web institucional y el aula virtual dentro de un mismo entorno visual, se mitiga el rechazo inicial del alumnado, sustituyendo los menús estáticos por mecánicas de exploración que fomentan de manera orgánica la autonomía y la madurez organizativa del estudiante desde etapas tempranas.

1.3 USUARIOS O DESTINATARIOS

Para justificar el volumen de trabajo y la envergadura exigida a un equipo de desarrollo compuesto por dos personas, el sistema define e interactúa con un ecosistema de destinatarios segmentado en tres perfiles con necesidades y roles claramente diferenciados:

- **Alumnado (Usuarios Activos):** Estudiantes del centro educativo que representan los usuarios principales de la aplicación. Su rol dentro del sistema es dinámico e interactivo; controlan un avatar personalizable con el que exploran de forma visual el mapa escolar en 2D. Sus necesidades críticas incluyen la centralización visual de sus calificaciones, alertas inmediatas de tareas pendientes, un calendario escolar intuitivo y una interfaz que traduzca las obligaciones académicas en objetivos visualmente comprensibles.
- **Profesorado y Personal de Administración (Usuarios de Gestión):** Personal técnico encargado de la gobernanza y mantenimiento de los contenidos informativos del sistema. El profesorado requiere un panel de control simplificado tipo formulario para realizar la inserción de calificaciones, programar eventos en el calendario y publicar avisos sin necesidad de interactuar con el motor gráfico de juego. El personal administrativo asume el rol de gestión de la base de datos de matriculaciones y la secretaría virtual.
- **Centros Educativos (Clientes):** Instituciones escolares de carácter público o privado que adquieren e implementan la solución web. Sus necesidades se centran en reducir la tasa de incidencias por pérdidas de acceso, disminuir la brecha digital en la comunicación con el alumnado, centralizar su infraestructura web bajo un único entorno escalable y elevar el compromiso formativo de su comunidad estudiantil a través de la innovación tecnológica.

1.4 TIPO DE PROYECTO PROPUESTO

El proyecto está planteado de manera estricta para una **solución web interactiva**. Se descarta el desarrollo de aplicaciones nativas de escritorio o de aplicaciones móviles

independientes con el objetivo de priorizar la compatibilidad absoluta y la accesibilidad universal dentro del entorno escolar.

La elección de una plataforma web (accesible bajo arquitectura cliente-servidor) se justifica técnicamente debido a que permite el acceso inmediato a la aplicación desde cualquier dispositivo dotado de un navegador web estándar (ya sean ordenadores de las aulas de informática del centro, tabletas o dispositivos personales en el hogar), eliminando las barreras de instalación, compilación y mantenimiento de software local en los equipos del instituto. Este formato resulta idóneo para solventar la necesidad detectada, ya que unifica bajo una única URL la funcionalidad del portal institucional y el aula virtual, permitiendo actualizaciones del lado del servidor en tiempo real y garantizando un rendimiento óptimo independientemente del hardware utilizado por el estudiante.

1.5 OBLIGACIONES LEGALES, NORMATIVAS Y FISCALES

Al tratarse de una solución web implantada en un entorno educativo que gestiona expedientes académicos e información de carácter sensible, el cumplimiento del marco legal vigente constituye un pilar central del diseño técnico del sistema:

- **Protección de Datos (RGPD y LOPDGDD):** El almacenamiento de calificaciones, nombres, apellidos y cursos requiere la máxima categoría de seguridad en el tratamiento de datos. El sistema debe implementar un cifrado de extremo a extremo mediante HTTPS y algoritmos de hash robustos para las credenciales. Al trabajar con datos de alumnos que pueden incluir a menores de 14 años, la plataforma debe contemplar obligatoriamente la gestión de perfiles de acceso supervisados y el consentimiento explícito de los tutores legales para el tratamiento informático de la información académica.
- **Seguridad y Roles de Acceso:** Para salvaguardar la integridad de los datos, el backend de la aplicación web debe aplicar de forma estricta un control de acceso basado en roles (RBAC). Se prohíbe que el rol "Alumno" posea permisos de escritura o modificación sobre las tablas de calificaciones o configuración del mapa, restringiendo estas operaciones exclusivamente a los tokens autenticados del profesorado o administradores.
- **Accesibilidad Web (Norma UNE-EN 301549 / WCAG 2.1):** Como software destinado a un servicio público educativo, la interfaz web debe garantizar la inclusión. El motor de juego 2D y los menús modales emergentes deben incorporar alternativas textuales, compatibilidad con lectores de pantalla y opciones de alto contraste para alumnos con diversidad funcional visual o motora.

- **Propiedad Intelectual y Licencias de Software:** El desarrollo se fundamenta en herramientas de código abierto (Open Source). Las licencias del software del servidor (Node.js/Express) y del framework del cliente (Phaser.js) se auditan para garantizar su gratuidad y uso educativo. Asimismo, los recursos gráficos bidimensionales utilizados para el diseño del mapa y los personajes se acogen a licencias de uso libre (Creative Commons), documentando debidamente su autoría en los créditos de la memoria técnica.

1.6 ALCANCE DEL PROYECTO Y GUIÓN DE TRABAJO

Alcance del Proyecto: El proyecto comprende el diseño conceptual, el modelado relacional y la implementación técnica de un prototipo funcional de aplicación web interactiva en 2D. El sistema incluye un servidor backend completo con API REST para la gestión de datos académicos y un cliente frontend interactivo basado en un motor de juego que renderiza el mapa del centro escolar, gestiona las colisiones físicas del avatar y despliega las interfaces de consulta.

Para asegurar la viabilidad del desarrollo dentro de los plazos académicos establecidos, el alcance del prototipo inicial se acota estrictamente a las siguientes delimitaciones:

- *Incluido:* Sistema de inicio de sesión con diferenciación de tres roles; renderizado de un mapa 2D compuesto por tres salas principales (Aula, Secretaría y conserjería); movimiento del avatar en cuatro direcciones; base de datos relacional operativa para la persistencia de calificaciones; y paneles de visualización interactiva para el alumno.
- *Excluido:* Queda fuera del alcance de esta memoria el desarrollo de pasarelas de pago para servicios de comedor, la gestión de nóminas o recursos humanos del profesorado, el modelado tridimensional (3D) de las instalaciones y el despliegue comercial en servidores de producción masiva para múltiples centros simultáneos.

Guión de Trabajo: La estructura secuencial de la presente memoria técnica se articula bajo las siguientes líneas de desarrollo:

1. *Análisis del sector, estudio de la problemática y viabilidad del entorno educativo (Bloque 1 - RA1).*
2. *Especificación formal de requisitos del sistema, modelado de la arquitectura cliente-servidor y diseño del esquema relacional de la base de datos (Bloque 2 - RA2).*

3. *Planificación temporal de la ejecución, asignación de responsabilidades de desarrollo, gestión de riesgos y valoración económica operativa (Bloque 3 - RA3).*
4. *Desarrollo técnico del código fuente, integración de módulos frontend/backend y resultados del cuaderno de pruebas de calidad (Bloque 4 - Desarrollo y Conclusiones).*

2. RA2. PLANIFICACIÓN TÉCNICA Y DISEÑO

2.1 JUSTIFICACIÓN

La selección y puesta en marcha de este proyecto se fundamenta en la necesidad imperativa de transformar la relación digital entre las instituciones educativas y su alumnado. Desde un punto de vista organizativo y social, los entornos virtuales tradicionales se han convertido en repositorios estáticos de archivos PDF que generan apatía en el estudiante, aumentando el riesgo de desvinculación escolar digital. Este proyecto aporta un valor diferencial al demostrar que las tecnologías habitualmente asociadas al entretenimiento interactivo pueden ser aplicadas con rigurosidad académica para optimizar los flujos de comunicación interna de un centro educativo.

Desde la perspectiva técnica, el proyecto se justifica al unificar bajo una arquitectura moderna de software web dos conceptos tradicionalmente disjuntos: un sistema de gestión académica relacional y un motor de renderizado interactivo bidimensional en tiempo real. Esta integración demuestra la utilidad potencial de la aplicación, ofreciendo a la comunidad educativa un entorno intuitivo que elimina las fricciones administrativas y transforma los procesos de consulta de información académica en una experiencia inmersiva y fluida para el usuario.

2.2 OBJETIVO GENERAL Y OBJETIVOS ESPECÍFICOS

Objetivo General: Diseñar, desarrollar e implementar un prototipo funcional de aplicación web interactiva basada en un entorno 2D gamificado para centralizar la gestión de la trayectoria académica, la consulta de calificaciones y la organización escolar del alumnado dentro de un centro educativo.

Objetivos Específicos:

- **OE-01:** Desarrollar un servidor backend robusto mediante Node.js y Express que exponga una API REST segura para la gestión, validación y distribución de los datos escolares según el rol del usuario autenticado.
- **OE-02:** Diseñar e implementar un esquema de base de datos relacional en MySQL que garantice la persistencia y la integridad referencial de los expedientes académicos, perfiles de usuario y coordenadas lógicas del entorno interactivo.
- **OE-03:** Programar una interfaz cliente interactiva utilizando el framework de desarrollo web Phaser.js que renderice el mapa del centro en dos dimensiones de forma fluida y gestione de manera precisa las físicas de movimiento y colisión del avatar del alumno.
- **OE-04:** Integrar de forma segura un sistema de control de accesos basado en roles (RBAC) para segmentar las operaciones de consulta (alumnos) y edición (profesorado) de datos de acuerdo con las normativas de protección de datos vigentes.
- **OE-05:** Validar el rendimiento, la usabilidad y la seguridad global de la aplicación web mediante la ejecución de un cuaderno de pruebas técnico estandarizado que garantice la viabilidad del prototipo.

2.3 ESTUDIO DE VIABILIDAD TÉCNICA, ECONÓMICA Y TEMPORAL

- **Viabilidad Técnica:** El proyecto es plenamente realizable con las capacidades y tecnologías actuales. Las competencias técnicas fundamentales para el desarrollo del backend (Node.js, Express y bases de datos SQL) y del frontend (HTML5, CSS3, JavaScript estándar) forman parte de los conocimientos técnicos adquiridos y consolidados a lo largo del ciclo formativo. La curva de aprendizaje específica requerida para la implementación del motor gráfico en Phaser.js se considera controlable y asumible mediante el uso de su documentación oficial y su API abierta, garantizando la viabilidad de unificar ambos entornos de código en una arquitectura unificada.
- **Viabilidad Económica:** El desarrollo se plantea bajo un enfoque de optimización absoluta de recursos. Al fundamentarse en tecnologías de código abierto (*Open Source*) con licencias que permiten su uso gratuito con fines educativos y de desarrollo, el coste de adquisición de software es nulo (0\$). Las herramientas de desarrollo, servidores de bases de datos y editores de código no requieren inversión financiera. Para limitar los costes de infraestructura de hardware a corto plazo y mantener la consistencia del diseño técnico, el prototipo se desarrolla acotado a un único centro educativo y a un volumen de datos controlado (un curso escolar), haciendo que el proyecto sea completamente asumible y sostenible sin dependencias financieras externas.

- **Viabilidad Temporal:** El calendario de ejecución técnica está estrictamente planificado para completarse en un plazo improrrogable de 3 meses (12 semanas). El análisis temporal demuestra la viabilidad del proyecto siempre que se mantenga la acotación definida en el alcance: simplificar la estructura inicial de la base de datos a un único curso académico representativo para agilizar las fases de pruebas. El cronograma asigna bloques de tiempo específicos y secuenciales para el desarrollo de la lógica, el diseño del mapa, el testeo y la corrección de errores, asegurando la entrega del software finalizado dentro del periodo académico estipulado.

2.4 REQUISITOS FUNCIONALES Y NO FUNCIONALES

Requisitos Funcionales (RF)

- **RF-01 (Gestión de Autenticación):** El sistema debe validar las credenciales de acceso de los usuarios mediante un formulario de inicio de sesión seguro en la web.
- **RF-02 (Control de Roles - RBAC):** El sistema debe discriminar las interfaces y permisos del usuario según tres roles definidos: Alumno (consulta e interacción), Profesor (edición de notas y avisos) y Administrador (gestión total).
- **RF-03 (Renderizado del Escenario 2D):** El cliente web debe cargar y renderizar en pantalla un mapa bidimensional en perspectiva aérea o lateral que simule la infraestructura física del centro escolar.
- **RF-04 (Control y Movimiento del Avatar):** El usuario con rol Alumno debe poder desplazar un personaje gráfico por el mapa 2D utilizando los controles estándar del teclado (flechas de dirección o combinación WASD).
- **RF-05 (Detección de Colisiones e Interacción):** El sistema debe detectar cuando el avatar colisiona o se posiciona sobre un punto de interés interactivo (Aulas, Secretaría, Conserjería) e iniciar automáticamente el evento correspondiente.
- **RF-06 (Módulo de Calificaciones):** Al interactuar con el área de "Aulas", el sistema debe desplegar una interfaz flotante (modal) que muestre de forma ordenada las asignaturas, notas y tareas pendientes extraídas del servidor.
- **RF-07 (Módulo de Secretaría Virtual):** El acceso del personaje a la zona de "Secretaría" debe permitir la visualización y descarga de circulares informativas, horarios y menús escolares de la institución.
- **RF-08 (Panel de Gestión Docente):** El sistema debe ofrecer una interfaz web independiente basada en formularios para que los usuarios con rol Profesor puedan registrar calificaciones y modificar la información académica sin interactuar con el motor gráfico.

Requisitos No Funcionales (RNF)

- **RNF-01 (Rendimiento y Tiempo de Respuesta):** Las solicitudes de movimiento del avatar y la respuesta del entorno gráfico en el navegador deben procesarse con una latencia inferior a 100 milisegundos para garantizar una experiencia visual fluida.
- **RNF-02 (Seguridad en la Comunicación):** Toda transferencia de información entre el navegador cliente y el servidor backend debe viajar cifrada utilizando obligatoriamente el protocolo seguro HTTPS/TLS.
- **RNF-03 (Seguridad en Almacenamiento):** Las contraseñas de los usuarios deben guardarse de forma irreversible en la base de datos aplicando un cifrado de hash robusto (algoritmo bcrypt) con sal (*salt*).
- **RNF-04 (Escalabilidad Arquitectónica):** El diseño del software debe ser modular, facilitando la incorporación futura de nuevas salas al mapa interactivo (ej. Biblioteca, Gimnasio) o nuevas tablas a la base de datos sin alterar el código del núcleo del sistema.
- **RNF-05 (Disponibilidad y Multiplataforma):** La aplicación web frontend debe ser compatible con los principales navegadores del mercado (Google Chrome, Mozilla Firefox, Microsoft Edge y Safari) sin requerir complementos adicionales.

2.5 DISEÑO DEL SISTEMA

La solución se estructura bajo el patrón de arquitectura de software **Cliente-Servidor**, dividiendo netamente las responsabilidades de la interfaz de usuario de la lógica de procesamiento de datos. Esta separación resulta crítica para permitir que dos desarrolladores trabajen de forma simultánea e independiente en los componentes del sistema.

El sistema se compone de tres capas fundamentales que interactúan de forma cíclica y asíncrona:

1. **Capa de Presentación e Interacción (Cliente Frontend):** Se ejecuta por completo en el navegador del usuario. Aloja el motor de juego encargado de la renderización del mapa 2D, el control del personaje y la captura de las órdenes físicas del alumno. Funciona de manera reactiva: cuando el usuario realiza una acción, esta capa no procesa la lógica académica, sino que empaqueta la instrucción y la envía al servidor.
2. **Capa de Lógica de Negocio (Servidor Backend):** Actúa como el intermediario y supervisor de la aplicación. Recibe las peticiones HTTP provenientes del cliente, verifica la autenticidad del token del usuario, valida que la acción solicitada cumpla con los permisos de su rol y procesa las reglas del centro educativo.

Una vez procesada la lógica, devuelve una respuesta estructurada en formato JSON hacia el cliente para su representación gráfica.

3. **Capa de Persistencia (Base de Datos Relacional):** Almacenada en un servidor dedicado. Custodia los expedientes, contraseñas, configuraciones y el estado del mapa, respondiendo únicamente a las consultas estructuradas (SQL) emitidas de forma segura por la capa del servidor backend.

2.6 DISEÑO DEL PROGRAMA / ARQUITECTURA / BASE DE DATOS / INTERFACES

Arquitectura del Software y Distribución de Módulos

Para optimizar el flujo de trabajo del equipo de dos integrantes, la aplicación divide su código en dos grandes bloques técnicos:

- **Módulo Frontend (Desarrollador A):** Programado con tecnologías estándar de la web (**HTML5, CSS3, JavaScript ES6**) utilizando el framework **Phaser.js** como motor gráfico principal. Este módulo implementa la arquitectura de escenas de Phaser para separar el flujo de pantallas (Escena de Carga, Escena de Menú, Escena del Mapa Activo). Gestiona la carga de la matriz de azulejos (*Tilemap*) que da forma al escenario 2D, el bucle de actualización del juego (*Update Loop*) para capturar el movimiento del teclado, el motor de físicas básicas (*Arcade Physics*) para gestionar las colisiones con los muros del centro, y las peticiones asíncronas (*fetch*) que se comunican con el exterior.
- **Módulo Backend (Desarrollador B):** Desarrollado sobre el entorno de ejecución **Node.js** utilizando el framework **Express** para la creación de una API REST. Este módulo se organiza bajo el patrón de diseño Arquitectura en Capas: Capa de Rutas (mapeo de las URLs de la API), Capa de Controladores (lógica de negocio y validación de roles) y Capa de Modelos (consultas directas a la base de datos). Implementa middleware intermedios para la verificación de sesiones y seguridad.

Diseño de la Base de Datos

Para dar estricto cumplimiento al rigor técnico requerido, se implementa un modelo relacional en **MySQL**. El esquema del prototipo inicial se compone de las siguientes entidades, llaves primarias (PK), llaves foráneas (FK) y restricciones:

- usuarios (Gestiona las credenciales globales del sistema):
 - `id_usuario` INT AUTO_INCREMENT (PK)
 - `username` VARCHAR(50) UNIQUE NOT NULL

- password_hash VARCHAR(255) NOT NULL
 - nombre VARCHAR(50) NOT NULL
 - apellidos VARCHAR(100) NOT NULL
 - rol ENUM('admin', 'profesor', 'alumno') NOT NULL
- alumnos (Entidad dependiente que extiende los datos específicos de los estudiantes):
 - id_alumno INT AUTO_INCREMENT (PK)
 - id_usuario INT (FK referenciando a usuarios.id_usuario con restricción ON DELETE CASCADE)
 - curso VARCHAR(20) NOT NULL
 - sprite_avatar VARCHAR(50) DEFAULT 'default_avatar.png'
 - posicion_x FLOAT DEFAULT 0.0
 - posicion_y FLOAT DEFAULT 0.0
- asignaturas (Listado de materias impartidas en el centro):
 - id_asignatura INT AUTO_INCREMENT (PK)
 - nombre_asignatura VARCHAR(100) NOT NULL
 - id_profesor INT (FK referenciando a usuarios.id_usuario para asociar al docente a cargo)
- calificaciones (Persistencia del expediente escolar y notas de las entregas):
 - id_calificacion INT AUTO_INCREMENT (PK)
 - id_alumno INT (FK referenciando a alumnos.id_alumno ON DELETE CASCADE)
 - id_asignatura INT (FK referenciando a asignaturas.id_asignatura ON DELETE CASCADE)
 - evaluacion VARCHAR(20) NOT NULL (Ej. 'Primera Evaluación')
 - nota DECIMAL(4,2) NOT NULL CHECK (nota >= 0.00 AND nota <= 10.00)
 - fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- puntos_interes (Almacena las coordenadas del mapa 2D asociadas a eventos lógicos):
 - id_punto INT AUTO_INCREMENT (PK)
 - nombre_zona VARCHAR(50) NOT NULL
 - coordenada_x FLOAT NOT NULL
 - coordenada_y FLOAT NOT NULL
 - modulo_destino VARCHAR(50) NOT NULL (Determina qué modal o vista HTML se despliega: 'aula', 'secretaria', 'conserjeria')

Diseño de Interfaces (UI/UX)

El entorno visual se estructura para romper la monotonía administrativa tradicional mediante dos interfaces principales:

1. **Interfaz de Exploración Interactiva:** Una ventana de lona HTML5 (*Canvas*) gestionada por Phaser.js donde se muestra el mapa 2D del centro en vista cenital con una estética pixel-art amigable. El estudiante ve su personaje y puede caminar libremente. Las zonas interactivas están delimitadas por baldosas especiales visibles en el suelo.
2. **Interfaz Documental Emergente (Modales HTML):** Al activarse una colisión en una zona interactiva, la aplicación web congela temporalmente el bucle de juego y despliega de manera superpuesta un cuadro de diálogo modal estilizado con CSS moderno. Esta interfaz presenta los datos en tablas limpias, listados estructurados y tipografías claras de alta legibilidad, aislando la información académica para que el alumno se concentre en sus notas o tareas sin distracciones visuales.

2.7 RECURSOS NECESARIOS

Recursos Humanos (Equipo de Desarrollo)

El proyecto requiere una estructura de trabajo colaborativa compuesta por dos perfiles técnicos especializados:

- **1 Desarrollador Frontend / Diseñador UI:** Responsable de la lógica del cliente web, la programación de la física del motor gráfico en Phaser.js, la maquetación de las interfaces de usuario modales y la integración estética del mapa 2D.
- **1 Desarrollador Backend / Administrador de Datos:** Responsable de la programación del servidor en Node.js, la exposición de los servicios API REST, la seguridad y control de accesos basados en roles, y el modelado, optimización y mantenimiento de la base de datos MySQL.

Recursos Software (Entorno de Desarrollo Abierto)

- **Entornos de ejecución y Frameworks:** Node.js (versión LTS) con Express para la arquitectura del servidor; Phaser.js (versión 3.x) como biblioteca del motor de juego para el navegador cliente.
- **Sistemas de Gestión de Bases de Datos:** MySQL Server para el almacenamiento relacional, junto con la herramienta de administración visual MySQL Workbench o phpMyAdmin.
- **Editores de Código y Herramientas Técnicas:** Visual Studio Code como entorno de desarrollo integrado (IDE), equipado con extensiones para depuración de JavaScript y control de Git.
- **Herramientas de Diseño Gráfico y Mapeado:** Tiled Map Editor (software de código abierto especializado en la creación de mapas basados en azulejos para

exportar escenarios 2D en formatos estructurados JSON); herramientas de edición pixel-art como Aseprite o Piskel para el tratamiento de los sprites de los avatares.

Recursos Hardware

- **Estaciones de Trabajo:** Dos ordenadores personales dotados de procesadores estándar, un mínimo de 8 GB de memoria RAM y sistemas operativos compatibles (Windows, Linux o macOS) capaces de ejecutar los entornos de desarrollo locales.
- **Infraestructura de Pruebas:** Dispositivos terminales diversificados (un ordenador de escritorio, un ordenador portátil y una tableta digital) dotados de conexiones a internet para validar el correcto renderizado de la solución web multiplataforma.

2.8 FINANCIACIÓN O PRESUPUESTO

Para dar estricto cumplimiento a la observación del tribunal, se realiza una separación rigurosa entre el **coste operativo real de desarrollo del prototipo académico** realizado por los dos integrantes del equipo y el **presupuesto hipotético de implantación comercial** en caso de un despliegue profesional a gran escala en un centro educativo.

Presupuesto Operativo de Desarrollo del Prototipo Académico

Este análisis económico valora el coste real de la ingeniería invertida para construir el software que se presenta en el TFG:

- **Coste de Personal Técnico (Ingeniería de Software):** * *Desarrollador Frontend (Integrante 1):* 240 horas de trabajo estimadas a lo largo de las 12 semanas, valoradas a un precio de mercado junior de 25\$/hora = **6.000\$**.
 - *Desarrollador Backend (Integrante 2):* 240 horas de trabajo estimadas a lo largo de las 12 semanas, valoradas a un precio de mercado junior de 25\$/hora = **6.000\$**.
- **Coste de Equipamiento y Materiales:** Amortización de equipos informáticos propios preexistentes = **0\$**. Licencias de software y motores de desarrollo (*Node.js, Express, MySQL, Phaser.js, Tiled*) de carácter Open Source = **0\$**.
- **Infraestructura de Alojamiento para Pruebas:** Despliegue de la API y base de datos de testeo en servidores en la nube utilizando capas gratuitas para desarrolladores (plataformas tipo Render, Railway o Supabase) = **0\$**.

- **Margen de Contingencia Operativa (5%):** Fondo de reserva destinado a cubrir posibles licencias puntuales de software gráfico o ampliación de recursos en la nube = **600\$**.
- **COSTE TOTAL OPERATIVO DEL PROTOTIPO ACADÉMICO: 12.600\$.**

Presupuesto Hipotético de Implantación Comercial (Producción Profesional)

Este presupuesto proyecta de manera razonada el coste económico que supondría para una institución educativa contratar externamente la instalación, despliegue y adquisición final de la plataforma a nivel corporativo:

- **Adquisición de Infraestructura Hardware Física:** Servidor local dedicado de alta disponibilidad para el centro, sistemas de alimentación ininterrumpida (SAI) y adecuación técnica de la red escolar = **10.000\$**.
- **Licenciamiento, Despliegue y Configuración Profesional:** Contratación de servicios de consultoría informática para la integración masiva del software con las bases de datos de matrículas históricas del centro, parametrización de asignaturas de todos los cursos y personalización gráfica completa de la planta física del instituto = **20.000\$ - 22.000\$**.
- **INVERSIÓN INICIAL DE IMPLANTACIÓN COMERCIAL: 32.000\$.**
- **Coste de Sostenibilidad y Soporte Técnico:** Contrato de mantenimiento preventivo anual, parches de seguridad del servidor web, renovación de certificados SSL y soporte ante incidencias críticas = **1.500\$ - 1.800\$ / anuales**.

2.9 CONTROL DE CALIDAD

El aseguramiento de la calidad del software diseñado se articula mediante una estrategia de validación técnica dividida en cuatro ejes de control fundamentales:

- **Auditorías de Seguridad de Datos:** Se implementarán filtros de control para verificar que todas las peticiones que llegan al backend requieran de una sesión autenticada. Se realizarán pruebas de inyección SQL dummy y de alteración de parámetros en las URLs para certificar que un usuario con rol Alumno reciba un error explícito (HTTP 403 Forbidden) si intenta realizar operaciones de escritura sobre las tablas de calificaciones.
- **Métricas de Rendimiento y Carga:** Se configurarán herramientas de monitorización (como los perfiles de rendimiento de las herramientas de desarrollador del navegador y tests de carga concurrentes en el servidor backend). Se establecerá como criterio de aceptación estricto que el motor gráfico de Phaser.js mantenga una tasa de refresco constante de 60 fotogramas

por segundo (FPS) y que los tiempos de respuesta de la base de datos ante consultas académicas no superen el umbral de los 100ms.

- **Validación Funcional Integrada:** Creación de una matriz de pruebas unitarias para cada endpoint de la API REST utilizando la herramienta Postman. Cada función del sistema (inicio de sesión, desplazamiento por el mapa, guardado de notas de los profesores) debe ser testeada de forma aislada e integrada para confirmar que la respuesta obtenida se ajusta con exactitud a la especificación de los requisitos funcionales.
- **Evaluación de Usabilidad y Calidad de Interfaz:** Realización de pruebas de caja negra y sesiones de uso con un grupo de control reducido (simulado dentro del entorno de validación). El objetivo es medir la facilidad con la que un alumno localiza sus calificaciones dentro del mapa interactivo en 2D en comparación con una interfaz de aula virtual tradicional, garantizando de forma objetiva que el entorno visual resulta intuitivo, ameno y libre de errores de navegación (*bugs* gráficos).

3. RA3. PLANIFICACIÓN DE LA EJECUCIÓN

3.1 FASES DEL PROYECTO

El ciclo de vida del proyecto se articula en torno a seis grandes etapas secuenciales e iterativas. Cada una de ellas posee una finalidad clara dentro del proceso de ingeniería de software para asegurar que la transición entre la idea inicial y el prototipo final sea ordenada y controlada:

- **Planteamiento:** Esta fase inicial tuvo como objetivo la conceptualización de la idea de partida. El propósito fue definir una alternativa viable que superase la rigidez interactiva de las páginas web y las aulas virtuales tradicionales. Tras debatir y descartar varios enfoques, se consensuó el desarrollo de un portal web basado en la metáfora visual de un mapa interactivo escolar con navegación de personajes, logrando un formato entretenido, accesible y adaptado para fomentar la autogestión académica.
- **Análisis:** En esta etapa se evaluó la viabilidad del concepto inicial y se delimitó de forma estricta el alcance técnico del sistema para un grupo de dos desarrolladores. Su finalidad fue formalizar la especificación de requisitos

(funcionales y no funcionales), modelar los casos de uso principales, estructurar los roles de acceso (alumnado, profesorado y administración) y analizar la integración lógica entre el entorno gráfico del usuario y el servidor central de datos.

- **Diseño:** Su finalidad fue elaborar la arquitectura técnica y visual del sistema antes de iniciar la escritura de código fuente. Se dividió en dos vertientes: por un lado, el diseño arquitectónico, que definió el protocolo de comunicación (API REST) y los contratos de intercambio de datos estructurados (JSON) entre cliente y servidor; por otro lado, el diseño de interfaces (UI/UX), enfocado en la distribución visual de los módulos modales, la estructuración relacional de las tablas de la base de datos y el bocetado conceptual del mapa del centro escolar en dos dimensiones.
- **Desarrollo:** Constituye la fase de implementación técnica activa en la que se encuentra inmerso el proyecto. El trabajo se divide en la codificación backend de la lógica del servidor (Node.js y Express) y la programación del motor del mapa 2D en el cliente (Phaser.js). Dentro de esta fase se integra el desarrollo y optimización de los elementos gráficos indispensables (creación de la matriz de azulejos o *tilemaps* de las instalaciones y el diseño de los *sprites* de los avatares), dado que es en la programación del motor web donde estos componentes adquieren su comportamiento lógico e interactivo definitivo.
- **Pruebas:** Esta fase tiene como propósito fundamental el control de calidad y la detección temprana de fallos técnicos. Se programan tests unitarios para verificar la integridad de las rutas de la API, pruebas de conexión asíncrona para asegurar que las órdenes enviadas por el personaje impacten correctamente en el servidor, pruebas de persistencia en la base de datos (guardado, actualización y borrado de calificaciones) y auditorías de rendimiento gráfico para depurar retrasos de carga (*lag*) o inestabilidades en la web.
- **Presentación:** La fase de cierre operativo del proyecto. Su finalidad es la consolidación de la memoria de TFG, la preparación de un entorno local estable o de demostración del software y la estructuración de la defensa pública, donde se expondrán los resultados del cuaderno de pruebas, la justificación de la viabilidad económica y el correcto funcionamiento de la plataforma web interactiva ante el tribunal evaluador.

3.2 SECUENCIACIÓN DE TAREAS

La planificación temporal y lógica de las actividades se rige por un principio de dependencia técnica estricta. No es viable desarrollar componentes visuales interactivos sin disponer previamente de una arquitectura de almacenamiento y un canal de comunicación firmemente estructurados. Por ello, la secuenciación de tareas se ha diseñado de acuerdo con la siguiente progresión razonada:

La primera tarea del equipo consistió en la puesta en común y unificación de criterios, donde se expusieron los puntos de vista individuales para concordar las bases de la aplicación. Concluido este bloque organizativo y de análisis de viabilidad, se dio paso inmediato a las tareas de creación gráfica preliminar e ingeniería de datos. El diseño de la apariencia base de la interfaz y la definición lógica de las tablas relacionales de la base de datos constituyeron el cimiento del proyecto.

Posteriormente, se activaron de forma simultánea las tareas de desarrollo técnico especializado: por una parte, la codificación de la lógica del servidor y la construcción de los *endpoints* de la API; por otra, la integración de la librería Phaser.js, la lógica de movimiento en el cliente y el desarrollo interactivo del escenario 2D. Esta simultaneidad en el desarrollo justifica la cooperación del grupo de dos alumnos, ya que ambos bloques de código debían respetar con exactitud los contratos de datos establecidos en la fase de diseño. El proceso concluye de forma lógica con las tareas de integración, pruebas cruzadas y resolución de errores (*bugfixing*), donde se testea la interacción del avatar con los puntos de interés del mapa y se corrigen las desviaciones de rendimiento antes de congelar la versión final de la aplicación web.

3.3 CRONOGRAMA

La distribución temporal del proyecto se ha estructurado de manera rigurosa a lo largo de un calendario de **3 meses (12 semanas)** de ejecución. Con el fin de atender la exigencia de coordinar un proyecto conjunto de dos personas, se desglosan las fases, los bloques semanales, las tareas específicas, el desarrollador responsable de su ejecución y los entregables tangibles asociados a cada hito:

Fase de Ejecución	Semana	Tareas Específicas a Desarrollar	Responsable	Entregable Asociado
Fase 1: Análisis y Requisitos	Semanas 1 - 2	Investigación del sector educativo (Moodle, Teams, Classroom). Modelado de casos de uso y especificación formal de Requisitos Funcionales (RF-01 a RF-08) y No Funcionales.	Ambos	Documento de Especificación de Requisitos de Software (SRS).
Fase 2: Diseño de Datos y API	Semanas 3 - 4	Diseño del esquema entidad-relación de la base de datos MySQL (tablas, llaves	Desarrollador B (Backend)	Diagrama de Base de Datos y Documentación

		primarias/foráneas). Definición y diseño de las rutas REST y contratos JSON del backend.		de Endpoints de la API.
Fase 3: Diseño de Interfaz UI/UX	Semanas 5 - 6	Configuración inicial del motor Phaser.js en el cliente. Creación conceptual del mapa del centro escolar en 2D y diseño base de los sprites de los avatares de usuario.	Desarrollador A (<i>Frontend</i>)	Esqueleto de Escenas en Phaser.js y Mockups de la Interfaz Web.
Fase 4: Desarrollo Backend	Semanas 7 - 8	Programación del servidor en Node.js y Express. Implementación del control de accesos basado en roles (RBAC) y lógica de persistencia de calificaciones académicas.	Desarrollador B (<i>Backend</i>)	Servidor API REST operativo y scripts SQL de población de tablas.
Fase 5: Desarrollo Frontend	Semanas 9 - 10	Importación del mapa de azulejos (<i>Tilemap</i>), programación del bucle de físicas de movimiento y colisiones del avatar. Programación de modales HTML para secretaría y notas.	Desarrollador A (<i>Frontend</i>)	Cliente Web interactivo con mapa 2D renderizado y conexión Fetch/Axios.
Fase 6: Integración y Pruebas	Semana 11	Conexión final cliente-servidor. Ejecución del cuaderno de pruebas unitarias, de rendimiento de datos (latencia < 100ms) y corrección de bugs de rendimiento.	Ambos	Cuaderno de Pruebas de Calidad técnico y Matriz de Errores Solventados.

Fase 7: Validación y Cierre	Semana 12	Pruebas de usabilidad de la interfaz web multiplataforma. Redacción final de la memoria técnica del TFG y preparación de la presentación pública.	Ambos	Código fuente consolidado en repositorio y Memoria Final del TFG.
--	--------------	---	--------------	---

3.4 PROCEDIMIENTOS DE TRABAJO

Para asegurar una ejecución ordenada, evitar conflictos en el desarrollo de software compartido y justificar de forma nítida la aportación técnica de cada integrante del grupo, se aplican los siguientes procedimientos metodológicos de trabajo:

- **Organización y Distribución de Roles:** El equipo se autogestiona bajo una metodología ágil simplificada. El **Desarrollador A** asume la responsabilidad total de la capa de cliente o Frontend (motores gráficos, renderizado 2D, animaciones, maquetación CSS de las interfaces académicas emergentes y usabilidad web). El **Desarrollador B** se responsabiliza en su totalidad de la capa del servidor o Backend (administración de la base de datos MySQL, optimización de consultas SQL, seguridad, encriptación de contraseñas mediante bcrypt y lógica de controladores en Node.js).
- **Gestión y Seguimiento de Tareas:** Se utiliza la herramienta colaborativa digital **Trello** mediante la implementación de un tablero Kanban estructurado en cuatro columnas: *Por hacer (Backlog)*, *En proceso*, *En fase de pruebas* y *Finalizado*. Cada tarea técnica se documenta en una tarjeta independiente que incluye el desarrollador asignado, la fecha límite y los requisitos técnicos de aceptación. Se realizan reuniones breves de sincronización semanales para evaluar los hitos alcanzados y desatascar posibles problemas técnicos de integración.
- **Gestión de Versiones de Código (GitFlow):** Todo el progreso del código fuente se centraliza en un repositorio privado de **GitHub**. Para garantizar la trazabilidad y la seguridad del desarrollo, se establece una política estricta de ramas. Queda terminantemente prohibido realizar aportaciones de código directamente sobre la rama principal (main). El Desarrollador A trabaja sobre la rama aislada feature/frontend y el Desarrollador B sobre feature/backend. Las unificaciones de código se realizan exclusivamente mediante solicitudes de cambio (*Pull Requests*) que deben ser revisadas, testeadas localmente de forma conjunta y aprobadas por ambos miembros del equipo para evitar la corrupción del software.

3.5 PLAN DE PREVENCIÓN DE RIESGOS

Durante el desarrollo del proyecto de la aplicación web interactiva pueden surgir desviaciones técnicas, organizativas o temporales. A continuación, se identifican los riesgos potenciales y se plantean medidas específicas de mitigación y contingencia:

- **Riesgo Técnico: Saturación o Caída de Rendimiento del Servidor Backend (Impacto: Alto / Probabilidad: Media):** El envío constante de eventos desde el motor gráfico Phaser.js (como solicitudes de movimiento, actualizaciones de coordenadas e interacciones simuladas simultáneas) puede saturar las peticiones HTTP a la base de datos de la aplicación.
 - *Medida de prevención/contingencia:* Acotar de forma estricta la base de datos del prototipo a un solo curso escolar controlado, limitando el volumen inicial de datos. Técnicamente, se implementará un control de eventos en el cliente (*throttling*) para espaciar las solicitudes y se optimizarán las consultas SQL mediante indexación para asegurar tiempos de respuesta inferiores a 100ms.
- **Riesgo Temporal: Desviación en el Calendario de Diseño Gráfico 2D (Impacto: Medio / Probabilidad: Alta):** La creación de la matriz de azulejos (*Tilemaps*) para simular los entornos físicos del centro educativo y las animaciones de los sprites de los avatares en cuatro direcciones puede absorber más tiempo del asignado en las semanas 5 y 6, retrasando la fase de desarrollo del código.
 - *Medida de prevención/contingencia:* Priorizar de forma estricta la funcionalidad lógica sobre la estética visual profunda. Durante las semanas de diseño se emplearán librerías de recursos de pixel-art gratuitos con licencias libres (Creative Commons). Los detalles estéticos personalizados y decorativos del centro se postergarán de forma exclusiva para la fase final de optimización en la semana 11.
- **Riesgo Organizativo: Conflictos críticos de Integración de Código (Impacto: Alto / Probabilidad: Baja):** Incompatibilidades técnicas u errores estructurales al unificar el código que gestiona la interfaz gráfica en Phaser.js con la API REST estructurada en Node.js, bloqueando el avance de las fases de pruebas.
 - *Medida de prevención/contingencia:* Firmar un contrato de interfaz estricto y definitivo en la semana 3, dejando asentadas las URLs de los *endpoints*, los parámetros requeridos y la estructura exacta del formato JSON de respuesta. Se realizarán pruebas de comunicación de datos asíncronas de manera simulada desde las primeras fases de desarrollo empleando software de testeado como Postman.

3.6 VALORACIÓN ECONÓMICA

Para dar respuesta rigurosa a las directrices de la memoria técnica, es preciso diferenciar de manera nítida el coste financiero de una implantación comercial a gran escala del **coste operativo real derivado de la ejecución de este prototipo académico**:

- **Costes de Personal Técnico (Dedicación Horaria):** El esfuerzo de desarrollo de este prototipo es asumido por los dos miembros del grupo. Estimando una carga de trabajo media de 20 horas semanales por integrante durante las 12 semanas del proyecto, se acumula un total de 480 horas técnicas de ingeniería de software. Computando un precio medio por hora de mercado junior de 25\$/hora, el esfuerzo operativo de personal se valora de la siguiente manera:
 - *Ingeniería Frontend y Diseño UI (Integrante 1):* 240 horas × 25\$/hora = **6.000\$**
 - *Ingeniería Backend y Datos (Integrante 2):* 240 horas × 25\$/hora = **6.000\$**
- **Costes de Equipamiento y Licencias de Software:** Para el desarrollo de este prototipo el coste financiero es de **0\$**. Se utilizan estaciones de trabajo informáticas personales preexistentes y amortizadas. Todo el ecosistema de programación empleado (*Node.js, Express, MySQL Server, Phaser.js, Visual Studio Code y Tiled*) se compone de herramientas de código abierto (*Open Source*) o licencias de carácter gratuito con fines educativos y de desarrollo.
- **Costes de Infraestructura de Pruebas en la Nube:** El despliegue de testeo y alojamiento temporal de la base de datos y la API web para las pruebas multiplataforma se realiza haciendo uso de la capa gratuita para desarrolladores de plataformas de hosting en la nube (como Render o Supabase), manteniendo este coste operativo controlado en **0\$**.
- **Margen de Contingencia Técnica (5%):** Se establece un fondo de reserva para mitigar posibles necesidades imprevistas durante la fase de ejecución, tales como la adquisición de alguna licencia de software de diseño gráfico complementaria o la ampliación puntual de potencia en los servidores de carga = **600\$**.
- **VALORACIÓN OPERATIVA TOTAL DE LA EJECUCIÓN DEL PROTOTIPO:**
12.600\$.

(Nota: Este coste representa la valoración económica del esfuerzo humano y técnico requerido para la fabricación del software del TFG, el cual difiere del presupuesto hipotético de 32.000\$ que requeriría una empresa externa para la adquisición de servidores físicos locales y la implantación comercial de la plataforma en las infraestructuras globales de un instituto).

3.7 DOCUMENTACIÓN DE EJECUCIÓN

El desarrollo técnico de la plataforma web interactiva queda respaldado y auditado mediante un conjunto de documentos técnicos y registros que garantizan la trazabilidad absoluta del proceso de ingeniería:

1. **Actas de Reunión y Sprint:** Documentos internos de carácter quincenal que sintetizan los acuerdos alcanzados por los dos miembros del equipo al término de cada ciclo Kanban. Reflejan el estado de las tareas de Trello, el grado de cumplimiento individual y la asignación de responsabilidades para el siguiente bloque del cronograma.
2. **Historial Técnico de Cambios (Repository Changelog):** Registro digital automatizado e inmutable provisto por la plataforma GitHub. Contiene el listado completo de confirmaciones (*commits*), las descripciones de los cambios introducidos en el código del servidor o del cliente, las ramas de desarrollo fusionadas y la autoría específica de cada línea de código.
3. **Cuaderno de Pruebas de Calidad:** Documento técnico detallado que recopila la matriz de pruebas funcionales ejecutadas. Contiene las trazas de validación de los endpoints en Postman, las capturas de respuesta del control de accesos por roles (RBAC) ante intentos de vulneración de seguridad, y las gráficas de rendimiento que certifican la tasa de 60 FPS en el cliente y tiempos de respuesta inferiores a 100ms en el servidor.
4. **Manual Técnico de Despliegue y Configuración:** Documento final que describe la arquitectura física de la aplicación. Incluye las instrucciones paso a paso para la instalación de dependencias en Node.js, la configuración de variables de entorno seguras, el script SQL estructurado para la creación y población inicial de las tablas en MySQL, y los requisitos técnicos de compatibilidad para el correcto arranque del motor de juego 2D en navegadores cliente.